

Problema de Asignación Generalizada (GAP)

Tecnología Digital V — Diseño de Algoritmos — TP2

Santos de los Rios

Lucas Ojeda

Facundo Riedel

1. Introducción

Con el crecimiento del e-commerce, las empresas de logística buscan bajar los costos de la *primera milla*: el traslado de los paquetes desde cada vendedor hasta un punto de recepción de la red. A un vendedor chico no le conviene que le programen una visita de recolección, así que es él mismo quien acerca los paquetes a un **depósito**: un comercio de la red que, como actividad secundaria, los guarda hasta que la empresa de logística los retira de forma consolidada.

Hoy ThunderPack opera de forma descentralizada: cada vendedor elige a qué depósito llevar sus paquetes y puede cambiar sin aviso. Esto hace un uso ineficiente de la capacidad de la red, sobre todo en los picos de demanda (Black Friday, Hot Sale), y genera reclamos y costos extra. Por eso evalúa pasar a una modalidad centralizada, en la que a cada vendedor se le asigna un depósito fijo, respetando las capacidades y minimizando la distancia total recorrida. Las preguntas de negocio que motivan el trabajo son:

- **(P1)** ¿Alcanza la capacidad de la red de depósitos, o hace falta ampliarla para dar respuesta a todos los vendedores?
- **(P2)** ¿Existe una asignación en la que los vendedores recorran una distancia razonable?
- **(P3)** ¿Es viable una herramienta que entregue soluciones de buena calidad en *pocos minutos*, utilizable incluso en tiempo real durante picos de demanda?

Modelamos el problema como un **Problema de Asignación Generalizada** (GAP). Como el GAP es NP-difícil, descartamos los métodos exactos para la escala real ($n = 1100$ vendedores, $m = 310$ depósitos) y usamos métodos heurísticos. Implementamos en C++ dos heurísticas constructivas, varios operadores de búsqueda local encadenados en un VND y dos metaheurísticas que lo usan adentro (ILS y LNS), y los comparamos sobre instancias de *benchmark* y sobre la instancia real de ThunderPack.

2. Descripción del modelo

Sean $N = \{1, \dots, n\}$ el conjunto de los n vendedores y $M = \{1, \dots, m\}$ el de los m depósitos. Cada depósito $i \in M$ tiene capacidad c_i . Para un vendedor $j \in N$ y un depósito $i \in M$, d_{ij} es la demanda (unidades) y c_{ij} el costo de asignar j a i (en ThunderPack, la distancia que recorrería j). Usamos variables binarias $x_{ij} = 1$ si j se asigna a i . El GAP en su versión general es:

$$\text{mín} \quad \sum_{i \in M} \sum_{j \in N} c_{ij} x_{ij} \quad (1)$$

$$\text{s.a.} \quad \sum_{i \in M} x_{ij} = 1 \quad \forall j \in N \quad (2)$$

$$\sum_{j \in N} d_{ij} x_{ij} \leq c_i \quad \forall i \in M \quad (3)$$

$$x_{ij} \in \{0, 1\} \quad \forall i \in M, j \in N.$$

La restricción (2) asigna cada vendedor a exactamente un depósito y (3) respeta las capacidades. Equivalentemente, una solución es una colección de conjuntos $\Gamma_1, \dots, \Gamma_m \subseteq N$ disjuntos, con Γ_i los vendedores asignados a i .

Soluciones parciales y penalización. Según las capacidades y demandas, puede que no exista una asignación factible que cubra a todos los vendedores. En ese caso admitimos *soluciones parciales* y penalizamos a cada vendedor sin asignar. Con $c_{max} = \max_{i \in M, j \in N} c_{ij}$ (la máxima distancia posible), la función objetivo que evaluamos es

$$f(s) = \sum_{j \text{ asignado a } i(j)} c_{i(j)j} + 3c_{max} \cdot |\{j \in N : j \text{ sin asignar}\}|. \quad (4)$$

La penalización $3c_{max}$ por vendedor sin asignar hace que meterlo en cualquier depósito sea siempre mejor que dejarlo afuera, sin volver inviables las instancias en las que la capacidad total no alcanza.

Interpretación como grafo / asignación. Es un problema de asignación con capacidades (*bin packing* con costos): los vendedores son ítems con demanda d_j , los depósitos contenedores con capacidad c_i , y asignar j a i cuesta c_{ij} . Esto orienta las heurísticas (asignar primero lo más cercano o urgente) y los operadores locales (mover o intercambiar vendedores entre depósitos).

Modelo de datos (implementación). El código refleja directamente el modelo: **Deposito** (id, capacidad c_i); **Vendedor** (id, demanda d_j y **distancias**, un vector de pares (c_{ij}, i) **ordenado por distancia creciente**, para probar primero los depósitos más cercanos); **Solucion** (deposito_asignado, vendedores_por_deposito y costo_total, con métodos **asignar()**/**sacar()**); e **Instancia** (depósitos, vendedores y matriz de costos; su constructor lee el formato OR-Library).

3. Algoritmos implementados

Todos los métodos operan sobre la misma representación de **Solucion** y mantienen la factibilidad ((3)) en cada paso. Notamos cap_i a la capacidad libre del depósito i en la solución actual.

3.1. Heurísticas constructivas

Vecino más cercano. Recorre los vendedores en orden y asigna cada uno a su depósito factible más cercano, aprovechando que **distancias** ya está ordenada. Si ningún depósito tiene capacidad, el vendedor queda sin asignar. Los primeros vendedores ocupan los mejores depósitos y los últimos pueden quedar sin lugar.

Algoritmo 1 Heurística constructiva: vecino más cercano

```

1: function VECINOMASCERCANO(inst)
2:    $cap_i \leftarrow c_i \ \forall i$ ;  $sol \leftarrow \text{vacío}$ ;  $f \leftarrow 0$ 
3:   for cada vendedor  $j \in N$  en orden do
4:      $asignado \leftarrow \text{false}$ 
5:     for all  $(dist, i)$  en  $distancias_j$  (de menor a mayor) do
6:       if  $cap_i \geq d_j$  then
7:         asignar  $j$  a  $i$ ;  $f += dist$ ;  $cap_i -= d_j$ ;  $asignado \leftarrow \text{true}$ ; break
8:       end if
9:     end for
10:    if no  $asignado$  then  $f += 3c_{max}$ 
11:    end if
12:  end for
13:  return  $sol$ 
14: end function

```

Inserción por arrepentimiento. En cada paso, para cada vendedor sin asignar calcula su mejor y segundo-mejor depósito factible; el arrepentimiento es la diferencia entre ambos costos (o ∞ si solo hay uno factible). Asigna al vendedor con mayor arrepentimiento a su mejor depósito. La idea es atender primero a quien más pierde si no toma ya su mejor opción, para reducir las malas asignaciones forzadas al final.

Algoritmo 2 Heurística constructiva: inserción por arrepentimiento

```

1: function INSERCIÓN(inst)
2:    $cap_i \leftarrow c_i \forall i$ ;  $sol \leftarrow \text{vacío}$ ; todos los  $j$  sin asignar
3:   while algún  $j$  sin asignar tiene depósito factible do
4:     para cada  $j$  sin asignar:  $p_j^1$  ( $p_j^2$ ) = costo de su mejor (segundo mejor) depósito factible
5:      $regret_j \leftarrow p_j^2 - p_j^1$  (o  $\infty$  si solo tiene uno factible)
6:     elegir  $j^* = \arg \max_j regret_j$ ; asignarlo a su mejor depósito; actualizar  $cap$  y  $f$ 
7:   end while
8:   penalizar los  $j$  que quedaron sin asignar
9:   return  $sol$ 
10: end function

```

3.2. Operadores de búsqueda local

Relocate múltiple. Operador generalizado: mueve hasta q vendedores convenientes de un depósito i a otro depósito k (los de mayor ahorro que quepan en la capacidad libre de k). El caso $q=1$ es el relocate simple: mover un solo vendedor j , con cambio de costo $\Delta = c_{kj} - c_{ij}$, aceptado si $c_{kj} < c_{ij}$ y $d_j \leq cap_k$. Es uno de los dos operadores principales que encadena el VND.

Algoritmo 3 Operador *relocate múltiple* (primera mejora)

```

1: function RELOCATEMULTIPLE(sol, inst, q)
2:   for all par de depósitos  $(i, k)$  con  $i \neq k$  do
3:      $mover \leftarrow$  hasta  $q$  vendedores de  $\Gamma_i$  con  $c_{kj} < c_{ij}$ , los de mayor ahorro  $c_{ij} - c_{kj}$ , que quepan en  $cap_k$ 
4:     if  $mover \neq \emptyset$  then
5:       mover cada  $j \in mover$  de  $i$  a  $k$ ; actualizar  $f$ ; return true
6:     end if
7:   end for
8:   return false
9: end function

```

Swap múltiple. Operador generalizado: aplica hasta q intercambios sucesivos (el mejor par factible cada vez) entre un par de depósitos i_1, i_2 . El caso $q=1$ es el swap simple: intercambiar dos vendedores $j_1 \in \Gamma_{i_1}$ y $j_2 \in \Gamma_{i_2}$, con cambio de costo $\Delta = (c_{i_2 j_1} + c_{i_1 j_2}) - (c_{i_1 j_1} + c_{i_2 j_2})$, aceptado si $\Delta < 0$ y el intercambio respeta ambas capacidades. Es el otro operador principal del VND.

Algoritmo 4 Operador *swap múltiple* (primera mejora)

```
1: function SWAPMULTIPLE(sol, inst, q)
2:   for all par de depósitos ( $i_1, i_2$ ) con  $i_1 < i_2$  do
3:     hechos  $\leftarrow 0$ 
4:     while hechos  $< q$  y existe un intercambio factible con  $\Delta < 0$  do
5:       aplicar el par  $j_1 \in \Gamma_{i_1}, j_2 \in \Gamma_{i_2}$  de menor  $\Delta = (c_{i_2 j_1} + c_{i_1 j_2}) - (c_{i_1 j_1} + c_{i_2 j_2})$ ;
       actualizar cap, f; hechos  $+= 1$ 
6:     end while
7:     if hechos  $> 0$  then return true
8:     end if
9:   end for
10:  return false
11: end function
```

Ambos operadores generalizados aplican una estrategia fija de aceptación (recorren los pares y se quedan con el primero que mejora, eligiendo dentro de él los mejores movimientos): no tienen variante first/best. Esa distinción sí existe en las *versiones simples* ($q=1$, un solo vendedor / un solo par), que implementamos por separado con dos criterios de aceptación: primera mejora (*first improvement*, aplica el primer movimiento que mejora) y mejor mejora (*best improvement*, explora todo el vecindario y aplica el de mayor ganancia). En E2 (§5) comparamos las versiones simples (first vs best) contra los dos operadores generalizados; el VND encadena estos últimos en primera mejora.

Operadores auxiliares para instancias ajustadas. Cuando la capacidad escasea y quedan vendedores sin asignar, los movimientos anteriores no siempre alcanzan. Implementamos:

- **asignar_no_asignados**: intenta colocar a cada vendedor sin asignar en su depósito factible más cercano.
- **asignar_con_eyecion**: para colocar a un vendedor sin asignar j en un depósito k sin lugar, expulsa a un vendedor x de k y lo reubica en otro depósito k' con capacidad, si el balance de costo es favorable.
- **cyclic exchange (ciclo3)**: rota tres vendedores entre tres depósitos ($v_1 \rightarrow d_2, v_2 \rightarrow d_3, v_3 \rightarrow d_1$), liberando lugar mediante un reacomodo cíclico que *relocate* y *swap* (limitados a dos depósitos) no pueden alcanzar. Para que sea tratable, los depósitos destino se restringen a los más cercanos de cada vendedor y se aplica la primera mejora. Es decisivo cuando la capacidad escasea y todos los depósitos vecinos están llenos, de modo que ningún movimiento entre dos depósitos consigue descender.

3.3. Metaheurísticas: VND, ILS y LNS

VND (Variable Neighborhood Descent). Encadena todos los vecindarios de búsqueda local: aplica cada uno hasta agotarlo, en un orden fijo, y repite el ciclo completo mientras alguno haya mejorado, hasta alcanzar un óptimo local respecto de todos los vecindarios. La versión implementada recorre, en primera mejora: **asignar_no_asignados**, eyección, los dos operadores principales *relocate múltiple* y *swap múltiple* ($q=1 \dots 10$) y *cyclic exchange* de 3 depósitos. Es la subrutina de mejora local que usan ambas metaheurísticas.

Algoritmo 5 Metaheurística: VND (Variable Neighborhood Descent)

```
1: function VND(sol, inst)
2:   mejora  $\leftarrow$  true
3:   while mejora do
4:     mejora  $\leftarrow$  false
5:     while ASIGNARNOASIGNADOS(sol) do mejora  $\leftarrow$  true
6:     end while
7:     while EYECCIÓN(sol) do mejora  $\leftarrow$  true
8:     end while
9:     for q = 1 to 10 do
10:      while RELOCATEMULTIPLE(sol, q) do mejora  $\leftarrow$  true
11:      end while
12:      while SWAPMULTIPLE(sol, q) do mejora  $\leftarrow$  true
13:      end while
14:    end for
15:    while CICLO3(sol) do mejora  $\leftarrow$  true
16:    end while
17:  end while
18: end function
```

ILS (Iterated Local Search). Parte de la mejor de las dos constructivas, ya mejorada con VND. En cada iteración copia la mejor solución y la perturba: aplica **fuerza** movimientos factibles al azar entre depósitos (mitad *swaps* y mitad *relocations*) para escapar del óptimo local. Después la reevalúa, le aplica VND y se queda con la mejor. Parámetros: **fuerza** de la perturbación, **semilla** y presupuesto de tiempo.

Algoritmo 6 Metaheurística: ILS (Iterated Local Search)

```
1: function ILS(inst, fuerza, semilla, segundos)
2:   mejor  $\leftarrow$  mejor de {VECINOMASCERCANO, INSERCIÓN}; VND(mejor); evaluar f(mejor)
3:   while tiempo transcurrido < segundos do
4:     s  $\leftarrow$  mejor; PERTURBAR(s, fuerza) [mitad swaps, mitad relocations al azar]
5:     evaluar f(s); VND(s)
6:     if f(s) < f(mejor) then mejor  $\leftarrow$  s
7:     end if
8:   end while
9:   return mejor
10: end function
```

LNS (Large Neighborhood Search / *ruin-and-recreate*). Misma estructura, pero con una perturbación más grande: en cada iteración desasigna **destruir** vendedores al azar (*ruin*) y deja que el VND los reinserte y reoptimice (*recreate*), aceptando solo si mejora. Al reconstruir desde cero una parte de la solución, LNS reordena asignaciones que ILS, atado a movimientos locales, no alcanza, y eso es clave en las instancias ajustadas (ver §5). Parámetros: **destruir**, **semilla** y presupuesto de tiempo.

Algoritmo 7 Metaheurística: LNS (Large Neighborhood Search)

```
1: function LNS(inst, destruir, semilla, segundos)
2:   mejor  $\leftarrow$  mejor de {VECINOMASCERCANO, INSERCIÓN}; VND(mejor); evaluar
   f(mejor)
3:   while tiempo transcurrido < segundos do
4:     s  $\leftarrow$  mejor; desasignar destruir vendedores al azar de s [ruin]
5:     VND(s); evaluar f(s) [recreate]
6:     if f(s) < f(mejor) then mejor  $\leftarrow$  s
7:     end if
8:   end while
9:   return mejor
10: end function
```

4. Consideraciones de implementación y dificultades

- **Compilación y uso.** Código en C++ (c++17). Desde `src/`: `make` compila el ejecutable `gap_simulator` con `g++ -O2`, que se corre con `./gap_simulator <instancia> <salida>`. `make tests` compila la suite de experimentación, que se ejecuta con `./tests <modo>` (modos: `check`, `e1-e6`, `convergencia`, `sinteticas`) y emite los CSV de `src/resultados/`; las figuras se generan con `python src/plots.py`.
- **Optimizaciones.** Distancias preordenadas por vendedor (pares (c_{ij}, i)) para probar primero los depósitos más cercanos; evaluación incremental del costo por Δ en $O(1)$ por movimiento (`evaluar` completo solo tras perturbar en ILS); y capacidad libre recalculada una vez por barrido.
- **No asignados y factibilidad.** Las soluciones parciales y la penalización $3c_{max}$ son centrales en las instancias ajustadas (b, e) y motivaron los operadores de eyección y de movimiento múltiple.
- **Uso de IA.** Usamos asistencia de IA dentro de lo habilitado por el enunciado. Para no ocupar lugar en el informe, adjuntamos un documento aparte con el detalle del uso que hicimos de la IA.
- **Dificultades.** (i) Bajo d_j varios grupos quedan infactibles (demanda > capacidad), lo que obligó a tratar soluciones parciales; (ii) el *swap múltiple* es $O(n^2)$ por barrido y pesa en la real ($n = 1100$); (iii) `destruir` resultó muy sensible y `fuerza` casi nada, lo que llevó al corte por tiempo.

5. Experimentación

5.1. Entorno de ejecución

- **Procesador / RAM / SO:** Ryzen 7 3800XT / 32gb / WSL
- **Compilador:** g++ 13.3.0.
- **Medición:** `std::chrono` alrededor de cada método. Las heurísticas y la búsqueda local son deterministas (una corrida); las metaheurísticas se promedian sobre $R = 5$ semillas y se cortan por tiempo según el tamaño (2/10/60s para chicas/medianas/grandes).
- **Reproducibilidad:** la experimentación está automatizada en `src/tests.cpp` (ver *Compilación y uso*); produce los CSV de `src/resultados/` y las figuras.

5.2. Instancias

Se usan tres grupos de *benchmark* del GAP de la OR-Library (tipos a , b , e) y la instancia real de ThunderPack. La Tabla 1 resume sus dimensiones.

Tabla 1: Dimensiones y resultados por instancia. m depósitos, n vendedores; capacidad y demanda totales bajo la lectura d_j (demanda del primer depósito). Columnas de costo: VMC (vecino más cercano), Inser. (inserción por arrepentimiento) y VND son corridas únicas determinísticas; ILS es el costo *medio* sobre 5 semillas (corte por tiempo según tier).

Instancia	m	n	Cap. total	Dem. total	VMC	Inser.	VND	ILS
<i>Grupo a</i>								
a05100	5	100	1710	1535	1.719	1.694	1.694	1.694
a05200	5	200	3355	3102	3.259	3.235	3.235	3.235
a10100	10	100	1920	1474	1.370	1.358	1.358	1.358
a10200	10	200	3650	2979	2.657	2.623	2.623	2.623
a20100	20	100	2000	1437	1.177	1.158	1.157	1.157
a20200	20	200	3980	3020	2.369	2.340	2.340	2.340
<i>Grupo b</i>								
b05100	5	100	1045	1440	5.669	5.473	5.461	5.354
b05200	5	200	2465	3127	8.703	9.106	8.396	8.381
b10100	10	100	1310	1547	3.461	4.417	3.324	3.172
b10200	10	200	2430	2897	7.551	8.373	7.179	7.010
b20100	20	100	1360	1517	3.157	3.269	2.605	2.446
b20200	20	200	2620	2865	4.945	5.980	4.491	4.337
<i>Grupo e</i>								
e05100	5	100	880	978	23.799	40.612	22.689	22.661
e05200	5	200	1727	1994	57.228	137.502	53.104	52.793
e10100	10	100	860	978	24.261	61.077	21.079	18.174
e10200	10	200	1684	1994	61.811	148.318	61.127	60.847
e10400	10	400	3412	4423	249.218	410.505	248.579	248.566
e20100	20	100	1111	978	3.791	2.420	2.420	2.393
e20200	20	200	1700	1994	54.165	145.766	50.835	50.026
e20400	20	400	3379	4423	232.167	419.183	231.056	231.055
e15900	15	900	7569	9712	620.956	818.881	617.389	617.383
e30900	30	900	7569	9712	537.376	902.207	536.413	536.305
e60900	60	900	7527	9712	478.813	1.023.132	465.466	465.407
e40400	40	400	3361	4423	180.792	506.013	173.226	172.820
e201600	20	1600	13438	17099	1.068.989	1.567.647	1.065.533	1.065.498
e401600	40	1600	13414	17099	1.012.219	1.600.339	1.010.194	1.010.137
e801600	80	1600	13417	17099	931.184	1.666.354	927.685	927.550
real	310	1100	15235	10707	788,2	718,0	695,5	683,6

5.3. Métricas

- **Costo** $f(s)$: función objetivo (4) (incluye penalización).
- **No asignados**: cantidad de vendedores sin depósito (mide factibilidad, P1).
- **Tiempo** de ejecución (P3).
- **Gap** relativo $(f(s) - f^*)/f^* \cdot 100\%$ respecto del *mejor valor encontrado* f^* entre todos los métodos y semillas para esa instancia.

5.4. Preguntas experimentales

- **(RQ1)** ¿Qué constructiva da mejor punto de partida (calidad y % sin asignar)?
- **(RQ2)** ¿Cuánto aporta cada operador de búsqueda local, y first vs. best improvement?
- **(RQ3)** ¿Mejora el VND respecto de un único operador? ¿Importa el orden de vecindarios?
- **(RQ4)** ¿Qué parámetros de ILS (**fuerza**) y LNS (**destruir**) dan la mejor relación calidad/tiempo, y cómo se comparan ambas metaheurísticas?
- **(RQ5)** En conjunto: constructiva $\rightarrow$ VND $\rightarrow$ ILS/LNS, ¿cuánto se gana y a qué costo de

tiempo, según el tamaño de la instancia?

- **(RQ6, caso real)** ¿Qué calidad y tiempo se obtienen en la instancia de ThunderPack, y qué implica para P1–P3?

5.5. E1 — Heurísticas constructivas (RQ1)

Hipótesis. Esperábamos que la inserción por arrepentimiento le ganara al vecino más cercano, por anticipar las asignaciones forzadas del final. Los tiempos de las dos son despreciables ($< 0,01$ s en todas las instancias).

Los resultados (columnas VMC e Inser. de la Tabla 1) confirman la hipótesis solo en parte: la inserción gana en el grupo *a* (holgado) y en la instancia real, pero es peor en los grupos ajustados *b* y *e*, donde además deja más vendedores sin asignar (ver `e1_constructivas.csv`). Comprometer temprano a los vendedores de mayor regret consume la capacidad escasa y fuerza más penalizaciones. El vecino más cercano, en cambio, al ir llenando los depósitos más próximos conserva mejor la factibilidad cuando la capacidad no sobra.

5.6. E2 — Operadores de búsqueda local (RQ2)

Hipótesis. Esperábamos que *best improvement* redujera más por barrido que *first* (a más costo) y que los operadores generalizados aportaran sobre los simples. Partiendo de cada constructiva aplicamos cada operador hasta óptimo local (sobre las 28 instancias y ambas constructivas), comparando seis: *relocate* y *swap* simples (un vendedor / un par) en *first* y *best*, y los principales *relocate múltiple* y *swap múltiple* ($q=1 \dots 10$, de estrategia fija, sin variante *first/best*).

El resultado (Tabla 2) es claro: ninguna variante de *relocate* mejora (0%), ni la múltiple, porque las constructivas ya asignan a cada vendedor su depósito factible más barato. Todo el trabajo lo hace *swap* (hasta 29,9% en el mejor caso); entre *first* y *best* la mejora media es casi igual (1,50% vs 1,51%). El *swap múltiple* no supera al simple en forma aislada (1,49%): su ventaja recién aparece combinado en el VND y en instancias ajustadas (E3). El VND encadena los dos operadores generalizados en primera mejora.

Tabla 2: Mejora (%) sobre el costo de la constructiva, tiempo y #pasadas, por operador y criterio de aceptación (promedio sobre todas las instancias y ambas constructivas). Los operadores *múltiple* aplican una estrategia fija ($q=1 \dots 10$), sin variante *first/best*.

Operador	Mejora media (%)	Mejora máx (%)	Tiempo medio (s)	# pasadas medias
<i>relocate</i> (first)	0,00	0,00	$< 0,001$	0,0
<i>relocate</i> (best)	0,00	0,00	$< 0,001$	0,0
<i>swap</i> (first)	1,50	29,91	0,007	24,2
<i>swap</i> (best)	1,51	29,04	0,013	15,4
<i>relocate</i> múltiple	0,00	0,00	$< 0,001$	0,0
<i>swap</i> múltiple	1,49	29,04	0,007	20,8

5.7. E3 — VND vs. operadores individuales (RQ3)

Hipótesis. Esperábamos que el VND, al combinar vecindarios, igualara o superara a cualquier operador individual, y que el VND *completo* fuera el más fuerte, sobre todo en las instancias ajustadas con vendedores sin asignar. Los operadores individuales de comparación son los simples *relocate* y *swap* (primera mejora).

Los datos (columna VND de la Tabla 1, frente a la mejor constructiva) confirman la hipótesis: el VND completo mejora de forma consistente a la mejor constructiva en las instancias ajustadas (p. ej. `b20100` $3,157 \rightarrow 2,605$, $\sim 17\%$; la real $718 \rightarrow 695,5$, $\sim 3,1\%$), porque la eyección y el *ciclo*

de 3 colocan vendedores que los movimientos de dos depósitos no pueden reubicar. En el grupo *a*, ya casi óptimo tras la constructiva, no hay margen y el VND apenas cambia el costo.

5.8. E4 — Calibración de parámetros: ILS y LNS (RQ4)

Como criterio de corte usamos el tiempo (no el número de iteraciones), escalado por tamaño: 2s para instancias chicas, 10s para medianas y 60s para grandes/real. Cada configuración se promedia sobre 5 semillas. Calibramos la **fuerza** de la perturbación de ILS y el tamaño de destrucción **destruir** de LNS sobre tres instancias representativas (una por tier, incluyendo casos ajustados *b/e*).

Hipótesis. ILS debería ser poco sensible a **fuerza** (con un óptimo intermedio) y LNS sensible a **destruir**. Esperábamos rendimientos comparables entre ambas.

Esa última expectativa no se cumple (Tabla 3): en las ajustadas LNS le gana a ILS por mucho (en **e15900**, $\sim 370,000$ vs $\sim 617,000$). El costo ahí es mayormente penalización por no asignados, que la destrucción-reconstrucción de LNS reduce más que las perturbaciones locales de ILS; en la real empatan (≈ 682). Sobre la calibración: ILS es casi plano en **fuerza**; LNS prefiere **destruir** chico en las ajustadas y grande en la real.

Tabla 3: Calibración: costo medio (5 semillas) según el parámetro. Mejor por fila en negrita. Corte por tiempo: 2 s (**b10200**), 10 s (**e15900**), 60 s (**real**).

<i>ILS — fuerza de perturbación</i>				
Instancia	5	10	20	40
b10200	7.020	7.015	7.010	7.017
e15900	617.389	617.379	617.383	617.375
real	685,3	684,0	683,6	683,6

<i>LNS — tamaño de destrucción destruir</i>			
Instancia	20	50	100
b10200	5.765	6.126	6.451
e15900	383.535	370.076	404.398
real	687,5	683,1	682,9

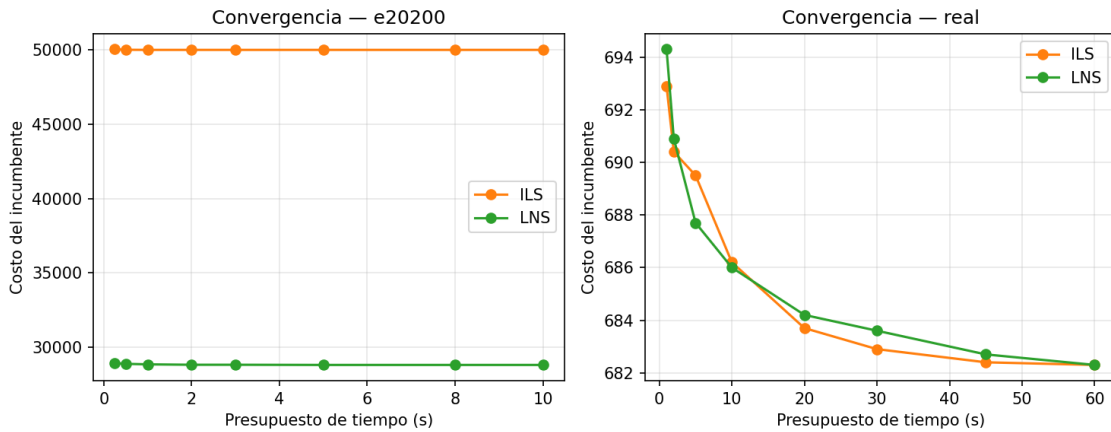


Figura 1: Convergencia: costo del incumbente vs. presupuesto de tiempo, para ILS y LNS, en una instancia ajustada (**e20200**) y en la real. LNS desciende más rápido en la ajustada; en la real ambas convergen a valores similares.

5.9. E5 — Comparación final (RQ5)

Hipótesis. Esperábamos que la calidad creciera de forma monótona constructiva $< \text{VND} < \text{metaheurística}$, con ganancias mayores en las instancias ajustadas, y que LNS marcara la diferencia justamente ahí. La Tabla 4 reporta el gap medio (%) de cada método respecto del mejor valor encontrado por instancia (menor es mejor), por grupo.

Tabla 4: Comparación final: gap medio (%) respecto del mejor valor encontrado por instancia, por método y grupo. Tiempo medio de la metaheurística (corte por tier). Menor gap es mejor.

Grupo	Constructiva	VND	ILS	LNS	Tiempo MH (s)
<i>a</i> (prom.)	0,0	0,0	0,0	0,0	2,0
<i>b</i> (prom.)	26,4	17,8	14,0	0,0	2,0
<i>e</i> (prom.)	94,3	88,9	86,7	0,0	17,4
real	5,2	1,9	0,0	0,0	60,1

El detalle por semilla (Tabla 5) confirma el patrón: en las ajustadas (*b*, *e*) LNS baja el costo medio muy por debajo de ILS (con más varianza, como en **e15900**); en la holgada **a05100** y en la real ambas empatan.

Tabla 5: ILS vs. LNS: costo sobre 5 semillas (mejor / peor / medio / desvío) en una instancia por grupo más la real. Menor es mejor; mejor costo medio por fila en negrita.

Instancia	ILS				LNS			
	Mejor	Peor	Medio	Desvío	Mejor	Peor	Medio	Desvío
a05100	1.694	1.694	1.694	0,00	1.694	1.694	1.694	0,00
b10200	6.996	7.022	7.009,80	9,68	5.735	5.864	5.764,80	50,20
e15900	617.375	617.389	617.383,20	5,38	351.670	423.046	387.206,20	27.002,29
real	682,30	684,80	683,56	0,95	682,30	683,70	682,90	0,52

Se confirma la hipótesis: la progresión constructiva $\rightarrow \text{VND} \rightarrow \text{MH}$ es monótona y LNS es el mejor método en casi todo el *benchmark* (gap 0%). ILS, atado a movimientos locales, no reinserta los no asignados y su gap medio en *e* llega a 86,7%; en el grupo *a* y en la real (holgados) todos convergen al mismo valor.

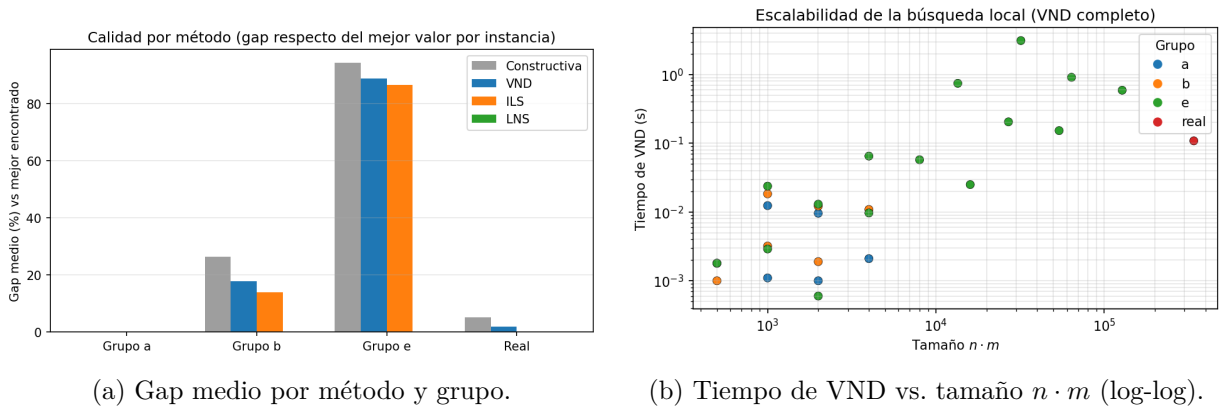


Figura 2: Resumen de la comparación final: calidad (gap) y escalabilidad de la búsqueda local.

5.10. E6 — Caso real ThunderPack (RQ6)

Lectura de negocio (P1–P3). La instancia real se resuelve con 0 vendedores sin asignar por todos los métodos: la capacidad agregada (ratio 0,70) alcanza y la geografía no lo impide, así

Tabla 6: Instancia real ($m = 310$, $n = 1100$): costo, vendedores sin asignar, distancia media por vendedor asignado y tiempo, por método (acumulado constructiva $\rightarrow$ VND $\rightarrow$ MH).

Método	Costo	Sin asignar	Distancia media	Tiempo (s)
Mejor constructiva	718,0	0	0,65	< 0,01
+ VND	695,5	0	0,63	0,10
+ ILS	682,3	0	0,62	60,1
+ LNS	682,3	0	0,62	60,2

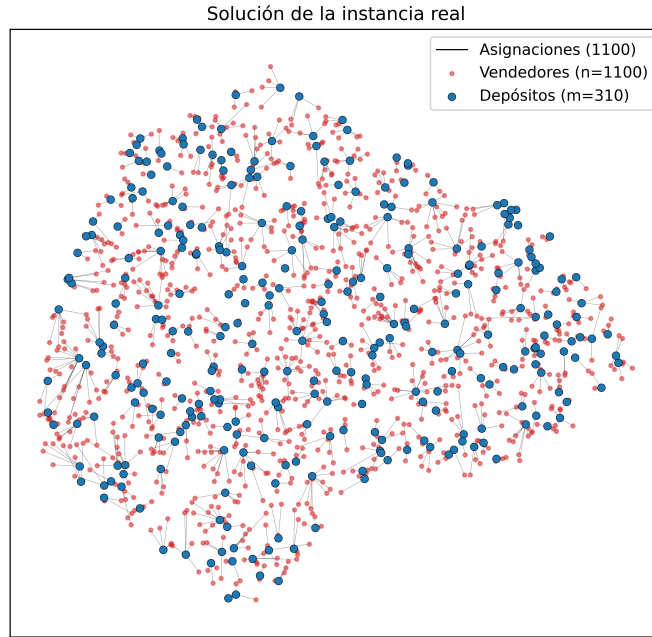


Figura 3: Disposición geográfica de la instancia real ($n = 1100$ vendedores en rojo, $m = 310$ depósitos en azul; datos de `real_instance.json`). Las líneas grises unen cada vendedor con el depósito al que fue asignado en la solución de ILS (leída de `solucion_competencia.txt`).

que la red no necesita ampliarse para la demanda actual (**P1**). La distancia media por vendedor baja de 0,65 (constructiva) a 0,62 (con ILS/LNS), o sea asignaciones cercanas y razonables (**P2**). En tiempo (**P3**), la constructiva es instantánea, el VND tarda $\sim 0,1$ s y ya recorta $\sim 3\%$, y las metaheurísticas en 60s ganan otro $\sim 2\%$. En menos de un minuto se obtiene una solución de buena calidad, viable como herramienta interactiva durante picos de demanda.

5.11. E7 — Peor y mejor caso (instancias sintéticas)

Para ver en qué casos cada método falla o acierta, además del *benchmark* construimos instancias mínimas *ad-hoc* (generadas dentro del archivo de *testing*) que disparan el peor y el mejor caso de cada tipo de método. La Tabla 7 resume lo observado.

En el caso *greedy miope*, el vecino más cercano asigna el primer vendedor a un depósito que el segundo necesitaba y queda en 101, mientras que el regret resuelve en 3. En el *óptimo local de 3 ciclos* se arma una asignación de la que ningún *relocate múltiple* ni *swap múltiple* (de dos depósitos) puede salir, pero el *cyclic exchange* de 3 del VND sí, y eso justifica ese operador. Por último, la metaheurística recupera el óptimo desde una constructiva muy mala, pero sobre una instancia ya óptima no puede mejorar: lo que gana depende de cuánto margen deje la búsqueda local.

Tabla 7: Peor/mejor caso sobre instancias sintéticas. En cada caso, el costo que obtiene cada método sobre la misma instancia.

Caso	Método	Costo	Interpretación
Greedy miope	Vecino más cercano	101	<i>peor caso</i> del greedy
	Inserción (regret)	3	<i>mejor caso</i> del regret
Óptimo local de 3 ciclos	relocate + swap múltiples	30	atascado (<i>peor caso</i> de 2 depósitos)
	VND completo (ciclo3)	3	escapa (<i>mejor caso</i> VND)
Metaheurística	ILS desde mala constructiva	3	recupera el óptimo (<i>mejor caso</i>)
	ILS sobre instancia óptima	2	sin mejora: solo <i>overhead</i> (<i>peor caso</i>)

6. Conclusiones, mejoras y observaciones

- **Respuesta a P1–P3 (caso real).** La red actual alcanza (0 sin asignar, no hace falta ampliarla, **P1**), con distancias cercanas (media 0,65 $\rightarrow$ 0,62, **P2**) y solución en menos de un minuto (**P3**); detalle en E6 (Tabla 6). En los grupos ajustados *b/e* la lectura d_j vuelve infactible asignar a todos.
- **ILS vs. LNS (hallazgo principal).** En instancias ajustadas LNS le gana a ILS por amplio margen, porque su destrucción-reconstrucción reubica vendedores no asignados que los movimientos locales de ILS no alcanzan; en la real (holgada) empatan. Recomendamos LNS por defecto, o un híbrido ILS+LNS.
- **Mejoras propuestas.** Criterio de aceptación tipo *simulated annealing*; **destruir** adaptativo en LNS; hibridar ILS+LNS; y generalizar el modelo a d_{ij} para comparar con los óptimos publicados.
- **Observaciones.** El *regret* no domina siempre (empeora en *b/e* bajo d_j); *relocate múltiple* no aporta sobre estas constructivas (todo el trabajo lo hacen *swap múltiple* y el *ciclo de 3*); y la brecha constructiva–metaheurística crece con lo ajustada que sea la instancia.

7. Solución entregada a la competencia

Para la competencia corrimos el *pipeline* completo (mejor constructiva $\rightarrow$ VND $\rightarrow$ metaheurística) sobre la instancia real de ThunderPack, dejándolo correr con un presupuesto de **una hora**. La mejor solución obtenida tiene **costo** 681,1 (la distancia total recorrida por los 1100 vendedores), con 0 **vendedores sin asignar** y una distancia media de $\approx$ 0,62 por vendedor. La asignación completa se entrega en el archivo `src/solucion_competencia.txt`: una línea por depósito con los identificadores de los vendedores asignados a ese depósito, en el formato del enunciado.